

✓ Baseline Notebook

✓ Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd
```

```
from utstd.folders import *
from utstd.iprenders import *
```

```
at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()
```

```
import warnings
warnings.simplefilter(action='ignore')
```

```
12.9/12.9 MB 47.3 MB/s eta 0:00:00
4.9/4.9 MB 61.3 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This b
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

✓ Student Information

```
group_name = "36106-26AU-AT3-Group 20"
student_name = "Devvrat Charusmiti Joshi"
student_id = "25657887"
```

```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 20

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Devvrat Charusmiti Joshi

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25657887

✓ 0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

O.b Import Packages

```
import pandas as pd
import altair as alt
import numpy as np

from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

alt.data_transformers.disable_max_rows()

DataTransformerRegistry.enable('default')
```

A. Assess Baseline Model

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Load data
try:
    X_train = pd.read_csv(at.folder_path / 'X_train.csv')
    y_train = pd.read_csv(at.folder_path / 'y_train.csv')

    X_val = pd.read_csv(at.folder_path / 'X_val.csv')
    y_val = pd.read_csv(at.folder_path / 'y_val.csv')

    X_test = pd.read_csv(at.folder_path / 'X_test.csv')
    y_test = pd.read_csv(at.folder_path / 'y_test.csv')
except Exception as e:
    print(e)
```

A.1 Generate Predictions with Baseline Model

```
target_column = "order_total"

# Convert target DataFrames to Series
y_train_model = y_train[target_column]
y_val_model = y_val[target_column]
y_test_model = y_test[target_column]

# Use prepared feature datasets
X_train_model = X_train.copy()
X_val_model = X_val.copy()
X_test_model = X_test.copy()

# Create baseline regression model
baseline_model = DummyRegressor(strategy="mean")

# Train baseline model
baseline_model.fit(X_train_model, y_train_model)

# Generate predictions
y_train_pred = baseline_model.predict(X_train_model)
y_val_pred = baseline_model.predict(X_val_model)
y_test_pred = baseline_model.predict(X_test_model)

baseline_prediction_value = baseline_model.constant_[0][0]

print("Baseline prediction value:")
print(round(baseline_prediction_value, 2))

print("\nTraining feature shape:", X_train_model.shape)
print("Validation feature shape:", X_val_model.shape)
print("Testing feature shape:", X_test_model.shape)
```

Baseline prediction value:
2253.43

Training feature shape: (10749, 39)
Validation feature shape: (2304, 39)
Testing feature shape: (2304, 39)

A.2 Selection of Performance Metrics

Provide some explanations on why you believe the performance metrics you chose is appropriate

```
def evaluate_regression_model(y_true, y_pred):
    """
    Calculates regression performance metrics.
    RMSE is calculated manually using sqrt(MSE) to avoid version issues
    """

    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    return mae, rmse, r2

# Calculate baseline metrics for train, validation, and test sets
train_mae, train_rmse, train_r2 = evaluate_regression_model(
    y_train_model,
    y_train_pred
)

val_mae, val_rmse, val_r2 = evaluate_regression_model(
    y_val_model,
    y_val_pred
)

test_mae, test_rmse, test_r2 = evaluate_regression_model(
    y_test_model,
    y_test_pred
)

baseline_metrics = pd.DataFrame({
    "dataset": ["Training", "Validation", "Testing"],
    "MAE": [train_mae, val_mae, test_mae],
    "RMSE": [train_rmse, val_rmse, test_rmse],
    "R2": [train_r2, val_r2, test_r2]
})

baseline_metrics
```

	dataset	MAE	RMSE	R2
0	Training	2036.199454	7011.410048	0.000000
1	Validation	1976.639830	5965.814564	-0.014636
2	Testing	1845.418877	4592.405192	-0.057243

```
baseline_prediction_preview = pd.DataFrame({
    "actual_order_total": y_val_model.values,
    "predicted_order_total": y_val_pred,
})

baseline_prediction_preview["error"] = (
    baseline_prediction_preview["actual_order_total"] -
    baseline_prediction_preview["predicted_order_total"]
)

baseline_prediction_preview["absolute_error"] = baseline_prediction_preview["error"].abs()

baseline_prediction_preview.head(10)
```

	actual_order_total	predicted_order_total	error	absolute_error
0	603.49	2253.428325	-1649.938325	1649.938325
1	6.28	2253.428325	-2247.148325	2247.148325
2	68.97	2253.428325	-2184.458325	2184.458325
3	603.49	2253.428325	-1649.938325	1649.938325
4	2341.97	2253.428325	88.541675	88.541675
5	4.99	2253.428325	-2248.438325	2248.438325
6	1228.83	2253.428325	-1024.598325	1024.598325
7	564.99	2253.428325	-1688.438325	1688.438325
8	69.99	2253.428325	-2183.438325	2183.438325
9	2404.97	2253.428325	151.541675	151.541675

```
performance_metrics_explanations = """
```

The selected performance metrics for the baseline regression model are MAE, RMSE, and R².

MAE, or Mean Absolute Error, is appropriate because the target variable order_total is a monetary value. MAE shows the average prediction error in the same unit as the target, making it easy to interpret from a business perspective. For example, an MAE of 500 means the model is wrong by about 500 currency units on average. RMSE, or Root Mean Squared Error, is also useful because the EDA showed that sales values are right-skewed and contain high-value orders. RMSE penalises large errors more strongly than MAE, so it helps show whether the model performs poorly on expensive or unusual orders. These metrics

```
# DO NOT MODIFY THE CODE IN THIS CELL
```

```
print_tile(size="h3", key='performance_metrics_explanations', value=performance_metrics_explanations)
```

```
performance_metrics_explanations
```

The selected performance metrics for the baseline regression model are MAE, RMSE, and R². MAE, or Mean Absolute Error, is appropriate because the target variable order_total is a monetary value. MAE shows the average prediction error in the same unit as the target, making it easy to interpret from a business perspective. For example, an MAE of 500 means the model is wrong by about 500 currency units on average. RMSE, or Root Mean Squared Error, is also useful because the EDA showed that sales values are right-skewed and contain high-value orders. RMSE penalises large errors more strongly than MAE, so it helps show whether the model performs poorly on expensive or unusual orders. These metrics

✓ A.3 Baseline Model Performance

Provide some explanations on model performance

```
# Add percentage-style error for extra interpretability
baseline_metrics_with_context = baseline_metrics.copy()

target_summary = pd.DataFrame({
    "dataset": ["Training", "Validation", "Testing"],
    "target_mean": [
        y_train_model.mean(),
        y_val_model.mean(),
        y_test_model.mean()
    ],
    "target_median": [
        y_train_model.median(),
        y_val_model.median(),
        y_test_model.median()
    ],
    "target_min": [
        y_train_model.min(),
        y_val_model.min(),
        y_test_model.min()
    ],
    "target_max": [
        y_train_model.max(),
        y_val_model.max(),
        y_test_model.max()
    ]
})

baseline_performance_summary = baseline_metrics_with_context.merge(
    target_summary,
    on="dataset",
    how="left"
)

baseline_performance_summary["MAE_as_%_of_mean_target"] = (
    baseline_performance_summary["MAE"] /
```

```
baseline_performance_summary["target_mean"] *  
100  
).round(2)
```

```
baseline_performance_summary["RMSE_as_%_of_mean_target"] = (  
    baseline_performance_summary["RMSE"] /  
    baseline_performance_summary["target_mean"] *  
    100  
).round(2)
```

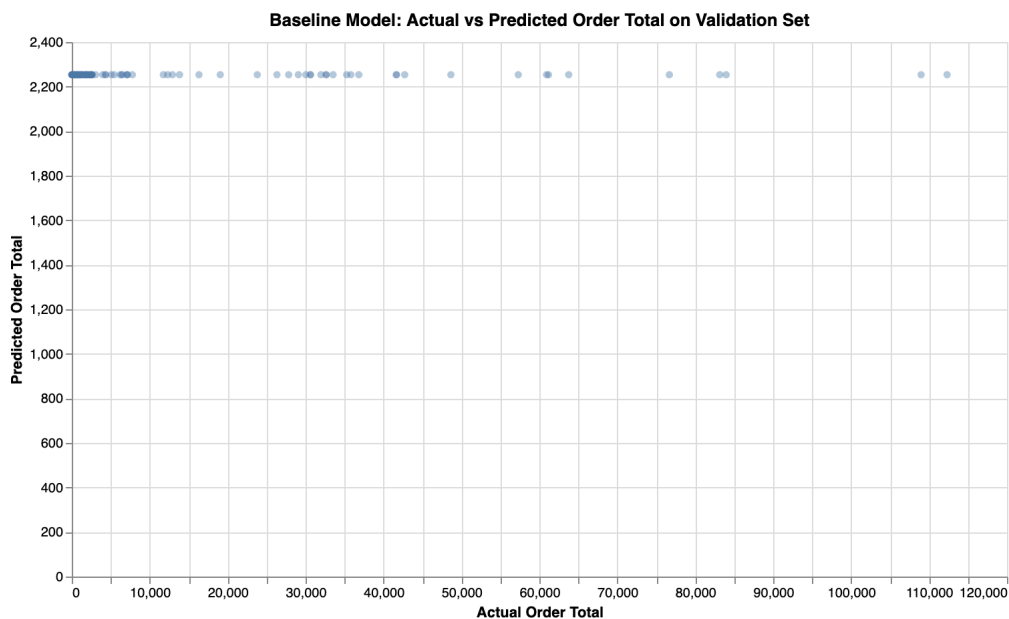
```
baseline_performance_summary
```

	dataset	MAE	RMSE	R2	target_mean	target_median	target_min	target_max	MAE_as_%_of_mean
0	Training	2036.199454	7011.410048	0.000000	2253.428325	1174.480	2.29	147390.932828	
1	Validation	1976.639830	5965.814564	-0.014636	1536.918902	588.960	2.29	112312.554000	
2	Testing	1845.418877	4592.405192	-0.057243	1184.834210	285.222	2.29	89869.276314	

```
# Visualise actual vs predicted values for validation set
```

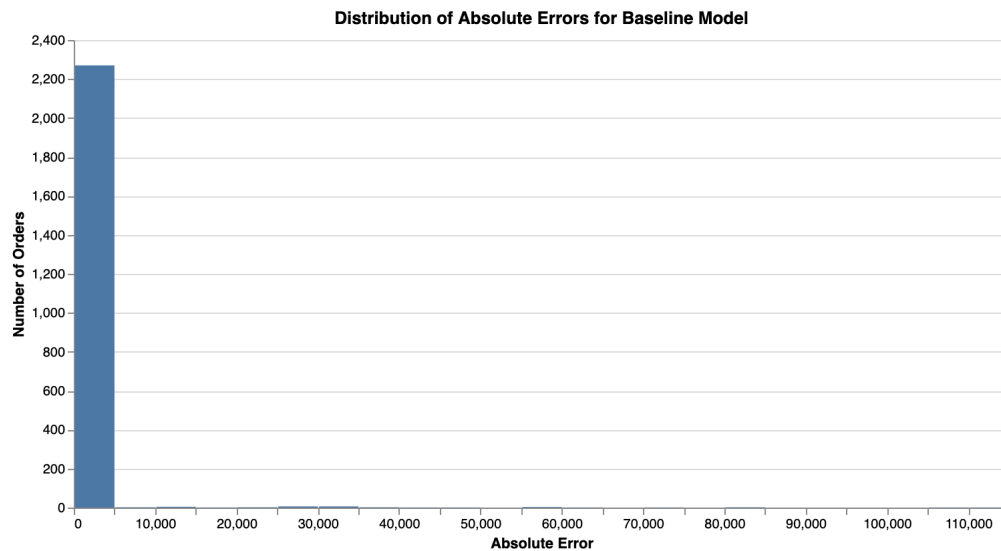
```
validation_plot_data = baseline_prediction_preview.copy()
```

```
alt.Chart(validation_plot_data).mark_circle(opacity=0.4).encode(  
    x=alt.X("actual_order_total:Q", title="Actual Order Total"),  
    y=alt.Y("predicted_order_total:Q", title="Predicted Order Total"),  
    tooltip=[  
        "actual_order_total",  
        "predicted_order_total",  
        "absolute_error"  
    ]  
).properties(  
    title="Baseline Model: Actual vs Predicted Order Total on Validation Set",  
    width=700,  
    height=400  
)
```



```
# Error distribution for validation set
```

```
alt.Chart(validation_plot_data).mark_bar().encode(  
    x=alt.X(  
        "absolute_error:Q",  
        bin=alt.Bin(maxbins=50),  
        title="Absolute Error"  
    ),  
    y=alt.Y("count():Q", title="Number of Orders"),  
    tooltip=["count()"]  
).properties(  
    title="Distribution of Absolute Errors for Baseline Model",  
    width=700,  
    height=350  
)
```



```
baseline_performance_explanations = ""
```

The baseline model uses DummyRegressor with the mean strategy. This means the model predicts the same value for every order: the average order_total from the training data. It does not use transaction, customer, person, territory, channel, or time-based features. This baseline is intentionally simple. Its purpose is to create a minimum performance level that future regression models must beat. If a later model cannot perform better than this baseline, then it is not learning useful patterns from the input features. The baseline model is expected to have relatively high MAE and RMSE because order_total is right-skewed. Many orders have smaller values, while some orders have very large totals. Since the baseline predicts only one average value, it will overpredict many low-value orders and underpredict many high-value orders. Overall, the baseline result provides a comparison point for later experiments. A useful regression model should

```
# DO NOT MODIFY THE CODE IN THIS CELL
```

```
print_tile(size="h3", key='baseline_performance_explanations', value=baseline_performance_explanations)
```

```
baseline_performance_explanations
```

The baseline model uses DummyRegressor with the mean strategy. This means the model predicts the same value for every order: the average order_total from the training data. It does not use transaction, customer, person, territory, channel, or time-based features. This baseline is intentionally simple. Its purpose is to create a minimum performance level that future regression models must beat. If a later model cannot perform better than this baseline, then it is not learning useful patterns from the input features. The baseline model is expected to have relatively high MAE and RMSE because order_total is right-skewed. Many orders have smaller values, while some orders have very large totals. Since the baseline predicts only one average value, it will overpredict many low-value orders and underpredict many high-value orders. Overall, the baseline result provides a comparison point for later experiments. A useful regression model should